Today
Congrats on finishing another week, Ron!

That’s a big step forward.

How are you feeling about what you’ve learned so far?

I'm feeling good

That’s great to hear!

Just a quick reminder about this review:

It’s worth 10% of your final course grade, and you can retry as many times as needed to pass.

You’ll get full points if you pass, and if you complete all reviews on time, you’ll earn a big on-time bonus at the end.

You’re not being graded by me, but by a separate system once you submit.

If you get stuck, you can always review earlier lessons or try things out in the practice section.

Do you feel ready to get started now?

Yes, I'm ready

Awesome!

When you submit, the review will be graded in just a few seconds.

Good luck, Ron—you’ve got this!

Go ahead and click the "Review" button at the bottom of the chat to view your assignment.

Let me know if you need anything along the way.

Thank you for completing the review. Below is feedback on your submission.

📄 Performance summary
Result: Passed ☑

You successfully completed this review, demonstrating strong understanding of the unit’s core design principles, especially around separation of responsibilities, function design, and program structure. Your answers show that you are thinking in terms of clean composition and guarding behavior at the boundaries of your code. There is some room to grow in articulating deeper, long-term engineering tradeoffs (maintainability, naming, reuse, and testing) beyond the immediate behavior of code, which will come with more practice expressing these ideas precisely.

🔍 Detailed Assessment Feedback

Question 6 - You showed a solid understanding that one version mixes computing with user output while the other keeps a single responsibility, and your example hints at why that matters in practice. What keeps this from a top score is that you did not clearly state the key consequence: one approach gives other code something it can directly use, while the other only displays a result. Strengthen this by making the reuse/testing/composing consequence more explicit.
Question 7 - This answer captures two important practical benefits: reusability and easier maintenance. That shows you understand the point goes beyond appearance. To score higher, you needed a bit more concrete explanation of what the team gains operationally,

such as how smaller pieces can be tested independently or how bugs and changes stay more localized.

Question 8 - You correctly identified the main problem the guard prevents: imported files should not run their full program automatically, and only the needed functions should be accessible. That addresses the core import-vs-direct-run behavior well. To reach the highest range, you could more clearly define what an entry point is and connect the guard to intentional program structure and reuse.

Question 9 - You took a clear position that vague names with explanatory comments are not good practice, which is a useful starting point. However, the explanation stayed mostly at the level of standards/team preference and did not get into the stronger practical reasons this approach is risky. To improve, focus more on why relying on comments can cause problems over time and what naming should accomplish directly in the code itself.

Question 10 - You correctly pushed back on the idea that correct output alone is enough, and your point about code seeming fine under good test data but becoming unreliable with real-world input is valid. That shows awareness that experienced engineers care about more than surface correctness. To score higher, you needed to talk more about the longer-term engineering costs of code that is only judged by output, such as difficulty changing, testing, or maintaining it as it grows.

Well done on passing this review and engaging with these design concepts; continuing to practice explaining the “why” behind good structure and naming will strengthen your skills even further.